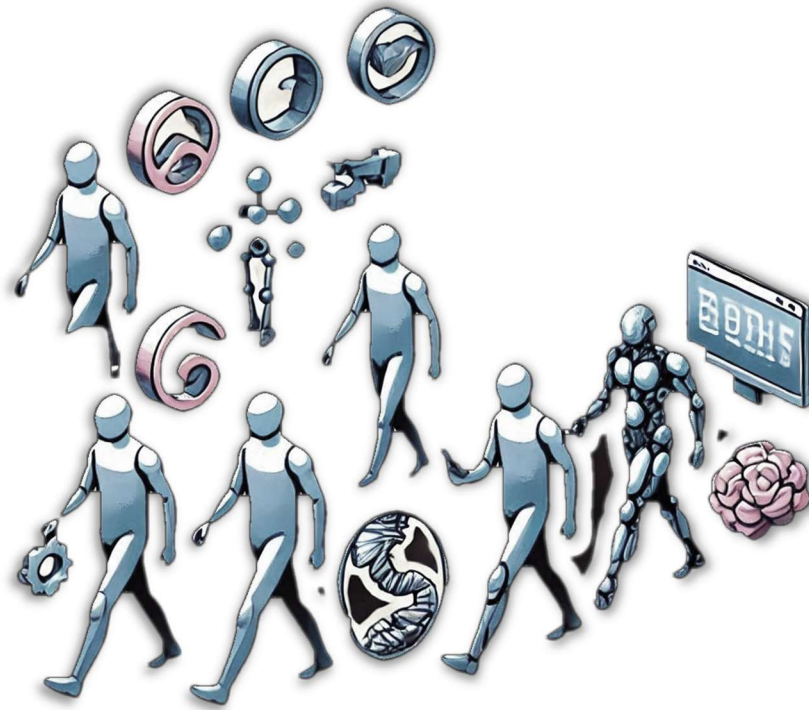


Capítulo 8

Polimorfismo



Fonte: Microsoft Designer – IA | Adaptada

8.1 Definição e Tipos de Polimorfismo



O polimorfismo é um dos conceitos centrais da orientação a objetos, permitindo que objetos de diferentes classes sejam tratados como se fossem do mesmo tipo. Ele promove flexibilidade e extensibilidade no código, tornando-o mais simples e reutilizável.

A palavra polimorfismo vem do grego e significa “muitas formas”. Em Orientação a Objetos, polimorfismo é a capacidade de um objeto ser referenciado por diferentes tipos, e de um método apresentar diferentes comportamentos dependendo do tipo do objeto que o invoca. Assim, um mesmo método pode gerar resultados variados, dependendo da classe do objeto que o implementa.

O polimorfismo é o quarto pilar essencial da Orientação a Objetos, ao lado de abstração, encapsulamento e herança. Este conceito permite que objetos de diferentes classes respondam de maneira distinta a uma mesma mensagem (método), tornando o código mais flexível, extensível e fácil de manter. Por meio do polimorfismo, podemos tratar diferentes objetos de forma uniforme, ao mesmo tempo em que cada objeto realiza uma ação específica de acordo com sua própria natureza. Neste capítulo, você compreenderá em profundidade o que é polimorfismo, como ele funciona na prática, aprenderá a implementá-lo em Java e a representá-lo graficamente por meio do diagrama de classe UML. Ao final, você será capaz de criar sistemas orientados a objetos mais elegantes, modulares e preparados para evoluir.

8.1.1 Exemplo do Mundo Real

Considere o ato de “dirigir”. Diferentes veículos (carro, moto, ônibus) podem responder ao comando “dirigir” de maneiras particulares, mas o motorista usa a mesma ação genérica. O “dirigir” é o mesmo verbo, mas sua implementação varia conforme o veículo.

8.1.2 Tipos de Polimorfismo:

- **Polimorfismo de Sobrescrita (Dinâmico):** Ocorre quando uma subclasse sobrescreve um método herdado da superclasse (conceito de herança), alterando seu comportamento. Dessa forma, o método é selecionado em tempo de execução com base no tipo real do objeto.

- **Polimorfismo de Sobrecarga (Estático):** Ocorre quando uma classe possui vários métodos com o mesmo nome, mas assinaturas diferentes (por exemplo, parâmetros de tipos ou quantidades distintas). A escolha do método ocorre em tempo de compilação, com base nos parâmetros fornecidos.

8.2 Implementação do Polimorfismo em Java



O polimorfismo é um dos conceitos fundamentais da Orientação a Objetos, permitindo que objetos de diferentes classes sejam tratados de maneira uniforme, desde que compartilhem uma mesma hierarquia de classes ou implementem uma interface comum. Em Java, o polimorfismo pode ser implementado de duas maneiras principais: polimorfismo em tempo de compilação (também chamado de *overloading*) e polimorfismo em tempo de execução (também conhecido como *overriding*). O *overloading* ocorre quando dois ou mais métodos em uma classe possuem o mesmo nome, mas diferentes assinaturas (quantidade ou tipo de parâmetros). Já o *overriding* acontece quando uma *subclasse* (classe filha) redefine um método da *superclasse* (classe pai) para personalizar seu comportamento.

Por exemplo, imagine que estamos desenvolvendo um sistema para um zoológico, onde temos uma classe base chamada *Animal* com um método chamado *emitirSom()*. A partir dela, criamos *subclasses* como *Cachorro* e *Gato*, cada uma implementando o método *emitirSom()* de forma diferente: o cachorro pode retornar "Latido", enquanto o gato retorna "Miau". Com o polimorfismo, podemos criar um único método que percorre uma lista de objetos do tipo *Animal* e chama *emitirSom()* para cada um deles, sem precisar saber de antemão se o objeto é um *Cachorro* ou um *Gato*. Isso permite que o código seja mais flexível, legível e fácil de manter, pois a lógica se adapta automaticamente ao tipo específico do objeto durante a execução do programa.

8.2.1 Polimorfismo no Contexto da Programação

Existem dois tipos principais de polimorfismo:

- **Polimorfismo em tempo de compilação:** Também conhecido como *sobrecarga de métodos*, ocorre quando vários métodos em uma classe têm o mesmo nome, mas assinaturas diferentes.

- **Polimorfismo em tempo de execução:** Também conhecido como *sobrescrita de métodos*, ocorre quando uma *subclasse* fornece sua própria implementação de um método definido na *superclasse*.

8.2.2 Polimorfismo de Sobrescrita

No exemplo da herança entre *Animal* e *Cachorro*, vimos que *Cachorro* sobrescreveu o método *emitirSom()* da superclasse *Animal*. Esse é um caso clássico de polimorfismo dinâmico: dependendo do tipo real do objeto referenciado, o método chamado terá comportamentos diferentes.

Código_8.1: Classe *AnimalPolimorfismo* em Java.

```
2 inheritors
1 public class AnimalPolimorfismo {
    2 overrides
    2 public void emitirSom() {
    3     System.out.println("Som genérico de animal.");
    4 }
    5 }
```

Código_8.2: Classe *CachorroPolimorfismo* em Java.

```
1 public class CachorroPolimorfismo extends AnimalPolimorfismo {
2     @Override
3     public void emitirSom() { System.out.println("Au au!"); }
6 }
```

Código_8.3: Classe GatoPolimorfismo em Java.

```

1 public class GatoPolimorfismo extends AnimalPolimorfismo {
2     @Override
3     public void emitirSom() {
4         System.out.println("Miau!");
5     }
6 }

```

Código_8.4: Classe PolimorfismoPrincipal em Java.

```

1 public class PolimorfismoPrincipal {
2     public static void main(String[] args) {
3         // Note que a variável é do tipo AnimalPolimorfismo
4         AnimalPolimorfismo a = new AnimalPolimorfismo();
5         AnimalPolimorfismo c = new CachorroPolimorfismo();
6         AnimalPolimorfismo g = new GatoPolimorfismo();
7
8         a.emitirSom(); // "Som genérico de animal."
9         c.emitirSom(); // "Au au!" (Cachorro)
10        g.emitirSom(); // "Miau!" (Gato)
11    }
12 }

```

Aqui, embora c e g sejam declarados como Animal, a execução do método *emitirSom()* depende do tipo real do objeto (Cachorro ou Gato). Esse é o polimorfismo em ação.

8.2.3 Polimorfismo de Sobrecarga

A *sobrecarga* ocorre quando criamos métodos com o mesmo nome na mesma classe, mas com parâmetros diferentes.

Código_8.5: Classe CalculadoraPolimorfismo em Java.

```
1 public class CalculadoraPolimorfismo {
2     // Soma de dois inteiros
3     public int somar(int a, int b) {
4         return a + b;
5     }
6
7     // Soma de três inteiros
8     public int somar(int a, int b, int c) {
9         return a + b + c;
10    }
11
12    // Soma de dois doubles
13    public double somar(double x, double y) {
14        return x + y;
15    }
16 }
```

Código_8.6: Classe CalculadoraPrincipal em Java.

```
1 > public class CalculadoraPrincipal {
2 >     public static void main(String[] args) {
3         CalculadoraPolimorfismo calc = new CalculadoraPolimorfismo();
4         System.out.println(calc.somar(a: 2, b: 3)); // 5
5         System.out.println(calc.somar(a: 2, b: 3, c: 4)); // 9
6         System.out.println(calc.somar(x: 1.5, y: 2.5)); // 4.0
7     }
8 }
```

Neste caso, a escolha de qualquer método somar é feita pelo compilador, com base nos argumentos passados (isso é polimorfismo estático).

8.2.4 Polimorfismo com Interfaces

Com interfaces, diferentes classes podem implementar o mesmo conjunto de métodos, permitindo que sejam tratadas de forma uniforme.

Exemplo:

Código_8.7: Implementação de interfaces em Java

```
1 // Abstração - interfaces
2 implementations
2 ⓘ interface FormaInterfaces {
3   implementations
3   ⓘ double calcularArea();
4 }
```

Código_8.8: Implementação de interfaces na subclasse em Java

```
1 class CirculoInterfaces implements FormaInterfaces {
2     private double raio;
3
4     > public CirculoInterfaces(double raio) { this.raio = raio; }
7
8     @Override
9     ⓘ > public double calcularArea() { return Math.PI * raio * raio; }
12 }
```

Código_8.9: Implementação de interfaces na subclasse em Java

```

1  class RetanguloInterfaces implements FormaInterfaces {
2      private double largura;
3      private double altura;
4
5      public RetanguloInterfaces(double largura, double altura){
6          this.largura = largura;
7          this.altura = altura;
8      }
9
10     @Override
11     public double calcularArea(){
12         return largura * altura;
13     }
14 }

```

Código_8.10: Implementação executada em Java

```

1  > public class TestaFormaInterfaces {
2  >     public static void main(String[] args) {
3      FormaInterfaces circulo = new CirculoInterfaces( raio: 2);
4      FormaInterfaces retangulo = new RetanguloInterfaces( largura: 3, altura: 4);
5
6      System.out.println("Área do círculo: " + circulo.calcularArea());
7      System.out.println("Área do retângulo: " + retangulo.calcularArea());
8  }
9  }

```


8.3 Representação do Polimorfismo em UML

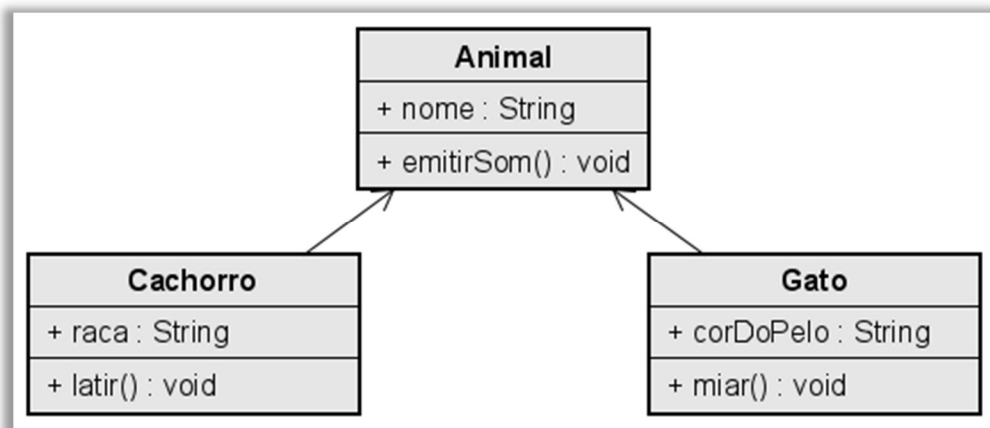


O polimorfismo fica evidente nos diagramas UML principalmente em dois casos:

- **Polimorfismo por Sobrescrita:** Ao representar herança, cada sub-classe que sobrescreve um método da superclasse mostra esse método com a mesma assinatura, indicando que ele produz comportamentos específicos.
- **Polimorfismo por Sobrecarga:** A classe com métodos sobrecarregados terá vários métodos com o mesmo nome, porém com parâmetros diferentes, mostrados no diagrama.

8.3.1 Exemplo UML (Sobrescrita):

Imagem_8.1: Diagrama de Classe UML representado as classes Animal, Cachorro e Gato.

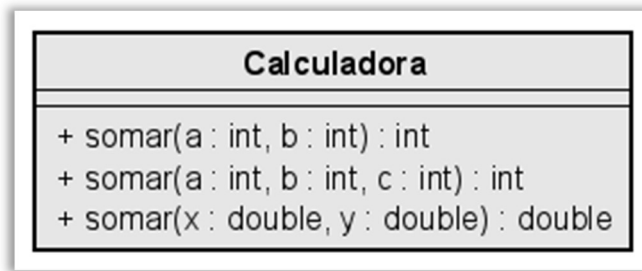


Nesse diagrama:

- Animal define o método emitirSom().
- Cachorro e Gato sobrescrevem esse método, cada um com seu próprio comportamento.

8.3.2 Exemplo UML (Sobrecarga):

Imagem_8.2: Diagrama de Classe UML representado a classe Calculadora.



A mesma classe Calculadora possui vários métodos `somar` com diferentes assinaturas.

Resumo do Capítulo



- **Polimorfismo:** Permite que o mesmo método seja usado por diferentes classes e objetos com comportamentos distintos (sobrescrita), ou que um método seja invocado de diferentes maneiras (sobrecarga).
- **Polimorfismo Dinâmico (Sobrescrita):** Definido em tempo de execução, a implementação específica do método é escolhida com base no tipo real do objeto.
- **Polimorfismo Estático (Sobrecarga):** Definido em tempo de compilação, a escolha do método depende da lista de parâmetros fornecida.
- **Benefícios do Polimorfismo:**
 - **Flexibilidade:** É fácil adicionar novas classes sem alterar o código que as utiliza.
 - **Extensibilidade:** Comportamentos distintos podem ser incorporados a um sistema sem complicações.
 - **Reutilização:** Pode-se escrever código genérico, deixando os detalhes para classes específicas.

Exercícios



- **Conceito de Polimorfismo:** Explique, com suas próprias palavras, o que é polimorfismo. Dê um exemplo de polimorfismo por sobrescrita e um exemplo de polimorfismo por sobrecarga.
- **Exemplo Prático:** Crie uma classe `Funcionario` com um método `calcularSalario()` que imprime um salário base. Em seguida, crie as subclasses `FuncionarioHorista` e `FuncionarioMensalista`, cada uma sobrescrevendo `calcularSalario()` para exibir um cálculo de salário diferente (por exemplo, $\text{hora} * \text{taxa}$ e salário mensal fixo).
- **Sobrescrita vs Sobrecarga:** Explique a diferença entre sobrescrita de métodos e sobrecarga de métodos. Como o compilador e a JVM lidam com cada caso?
- **Aplicação em Código:** Escreva uma classe `Operacoes` com métodos sobrecarregados `calcularArea()`: um para círculo (recebe o raio), outro para retângulo (recebe largura e altura) e outro para quadrado (recebe o lado). Teste esses métodos com diferentes valores.
- **Diagrama UML com Polimorfismo:** Desenhe um diagrama UML para representar uma hierarquia de `Forma` (superclasse), `Circulo` e `Retangulo` (subclasses), onde `Forma` possui um método `desenhar()` que é sobrescrito pelas subclasses. Adicione na mesma classe `Forma` um conjunto de métodos `calcularArea()`, sobrecarregados com parâmetros diferentes, para demonstrar polimorfismo por sobrecarga.

Conclusão



O polimorfismo é essencial para criar sistemas flexíveis e extensíveis. Ele permite que diferentes objetos sejam tratados de forma uniforme, reduzindo a complexidade e facilitando a manutenção do código.

O polimorfismo, aliado à abstração, encapsulamento e herança, forma a base de sistemas orientados a objetos poderosos e bem estruturados. Ao entender esses quatro pilares, você estará apto a projetar arquiteturas mais elegantes, robustas e fáceis de manter.

Com o entendimento sólido do polimorfismo, você possui agora as ferramentas fundamentais para criar sistemas orientados a objetos sofisticados e manter o código organizado, extensível e fácil de compreender. No próximo capítulo, aprofundaremos nossos conhecimentos, analisando o design orientado a objetos e boas práticas para criar arquiteturas escaláveis, coerentes e manuteníveis.